

3. **Next.js Commerce (Industry Alternative):**

- Vercel ka `next-js-commerce` starter template explore karein jo Next.js App Router aur Tailwind CSS ke sath Shopify Storefront API consume karta hai.

4. **Key Features Implement Karein:**

- Fast SSG/ISR (Static Site Generation) product pages ke liye.
- Client-side Cart state management (GraphQL mutations ke zariye cart update karna).
- Shopify Checkout redirect integration.

Step 4: Recommended Resources & Learning Path

Phase	Learning Focus	Best Resources
Phase 1	GraphQL Fundamentals	Official GraphQL Docs / Apollo GraphQL Tutorials
Phase 2	Shopify CLI & Remix Apps	Shopify Dev Documentation (App Tutorials)
Phase 3	Headless Commerce	Hydrogen Docs / Vercel Next.js Commerce GitHub repo
Phase 4	Build Practice Projects	Build 1 Custom Discount App & 1 Next.js Headless Storefront

Next Practical Goal

Pehla practical step yeh lein ke ek choti **Custom App** banayein jo Admin API ke zariye kisi product ka price automatically update kare, ya Next.js par Shopify Storefront API se products fetch karke ek simple custom store page display karein.

User prompt: Step 1: Base Technologies Prerequisites Shopify ki modern app ecosystem aur headless setups specific stack par chaltay hain: Remix / React: Shopify ka official app framework Remix (jo React-based hai) par built hai. React par aapka grip yahan bohot kaam aayega. Node.js & Express: Server-side logic, API handling, aur Database operations ke liye. GraphQL: Shopify Storefront aur Admin APIs dono GraphQL par heavily depend karti hain (REST APIs ab deprecated/secondary ho chuki hain). TypeScript: Professional app development aur headless setups mein standard hai. Step 3: Headless Commerce Roadmap Headless Commerce mein backend Shopify hota hai (for inventory, checkout, payment processing) aur frontend poora custom Next.js / Remix par hota hai. Shopify Storefront API: GraphQL queries ke zariye products, collections, cart, aur checkout URL fetch karna seekhein. Hydrogen & Oxygen (Shopify Native Stack): Hydrogen: Shopify ka official React/Remix-based headless framework. Oxygen: Shopify ki global hosting service jo Hydrogen stores ko host karti hai. Next.js Commerce (Industry Alternative): Vercel ka next-js-commerce starter template explore karein jo Next.js App Router aur Tailwind CSS ke sath Shopify Storefront API consume karta hai. Key Features Implement Karein: Fast SSG/ISR (Static Site Generation) product pages ke liye. Client-side Cart state management (GraphQL mutations ke zariye cart update karna). Shopify Checkout redirect integration. Headless sey hei start karty hey shopify mey

Response: Headless Commerce se start lena ek smart choice hai, kyunki aapka **React**, **Tailwind CSS**, aur **Shopify Theme Architecture** ka background is transition ko bohot smooth aur fast bana dega.

Neeche poora practical roadmap diya gaya hai jisse aap Step-by-Step pehla Headless Store build kar sakte hain.

Phase 1: Storefront API Access & Setup

Pehle Shopify Admin se Storefront API token generate karna hoga taake aapka frontend application store data ke sath communicate kar sake.

- 1. Development Store Create Karein:** Shopify Partner Account mein ek naya dev store banayein aur usmein 3–4 dummy products add karein.
- 2. Headless App Install / Create Karein:**
 - Shopify Admin → **Settings** → **Apps and sales channels** → **Develop apps**.
 - App name dein (e.g., `Nextjs-Headless-Frontend`).
 - **Configure Storefront API Scopes** par click karein aur `unauthenticated_read_product_listings`, `unauthenticated_write_checkouts`, `unauthenticated_read_content` allow karein.
- 3. Credentials Save Karein:**
 - **Public Access Token** aur **Storefront API Version** ko safe jagah copy kar lein.

Phase 2: GraphQL Fundamentals (Storefront API)

REST APIs ke xilaf, GraphQL mein aap wahi data demand karte hain jo screen par render karna hota hai. Pehle Shopify GraphQL App install karke dev store par yeh queries test karein:

1. Products Fetch Query:

```

query getProducts {
  products(first: 10) {
    nodes {
      id
      title
      handle
      featuredImage {
        url
        altText
      }
      priceRange {
        minVariantPrice {
          amount
          currencyCode
        }
      }
    }
  }
}

```

2. Cart Create Mutation:

```

mutation cartCreate($lines: [CartLineInput!]) {
  cartCreate(input: { lines: $lines }) {
    cart {
      id
      checkoutUrl
    }
  }
}

```

Phase 3: Project Architecture (Next.js App Router)

Next.js (App Router) + Tailwind CSS ke sath setup start karein:

1. Project Initialization:

```
npx create-next-app@latest my-headless-store --typescript --tailwind --app
```

2. Environment Variables (.env.local):

```

NEXT_PUBLIC_SHOPIFY_STORE_DOMAIN="your-store.myshopify.com"
NEXT_PUBLIC_SHOPIFY_STOREFRONT_ACCESS_TOKEN="your-public-access-token"

```

3. GraphQL Client Utility (lib/shopify.ts):

```

const domain = process.env.NEXT_PUBLIC_SHOPIFY_STORE_DOMAIN;
const accessToken = process.env.NEXT_PUBLIC_SHOPIFY_STOREFRONT_ACCESS_TOKEN;

export async function shopifyFetch<T>({ query, variables }: { query: string; variables?: object }): Promise<T> {
  const res = await fetch(`https://${domain}/api/2026-07/graphql.json`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-Shopify-Storefront-Access-Token': accessToken!,
    },
    body: JSON.stringify({ query, variables }),
    next: { revalidate: 3600 } // ISR Caching (1 Hour)
  });

  const json = await res.json();
  if (json.errors) throw new Error(json.errors[0].message);
  return json.data;
}

```

Phase 4: Core Features Implementation Order

Is Sequence ke hisab se components build karein:

1 Products Feed Page

HomePage / Catalog Grid

Next.js Server Components (RSC) use karke `shopifyFetch` Call karein aur Product Cards display karein. Tailored cards mein images, pricing, aur titles render karein.

2 **Dynamic Product Detail Page (PDP)**

App Route: `/products/[handle]`

`generateStaticParams` define karein taake tamaam product pages build time par **SSG** (Static Site Generation) ho jayen. Variant options (Color, Size) aur "Add to Cart" button render karein.

3 **Client-Side Cart Management**

React Context / Zustand

Cart items ko global state mein handle karein. Jab user "Add to Cart" par click kare, toh Shopify Storefront API ki `cartLinesAdd` mutation run ho aur `checkoutUrl` update ho.

4 **Checkout Redirect**

Native Shopify Checkout Integration

Custom Frontend par PCI compliance maintain karne ke bajaye user ko direct Storefront API se prapt hone wale `checkoutUrl` par redirect karein.

Headless Stack Comparison

Approach	Technology Stack	Hosting	Best For
Next.js Commerce	Next.js + React + Tailwind + Storefront API	Vercel	Maximum Flexibility, SEO Speed, Custom UI Frameworks
Shopify Hydrogen	Remix + React + Tailwind + Oxygen APIs	Shopify Oxygen	Native Shopify Ecosystem Integration & In-house Tools